

姓名	李泽浩	学号	2025218895	实验成绩	
专业班级	软件工程 25-9 班	实验时间	2026.6.14	实验地点	新安学堂

实验四：二叉树与树实验

1 实验目的

- 1) 掌握二叉树与树（森林）的存储表示。
- 2) 掌握二叉树与树（森林）的遍历算法（递归和非递归）。
- 3) 运用二叉树与树（森林）遍历算法求解有关问题。
- 4) 掌握树（森林）和二叉树的相互转换。

2 实验要求

- 1) 对用到的二叉树与树（森林）进行合理封装；
- 2) 在相应的存储结构上，实现基本运算；
- 3) 设计算法实现相应的实验任务；
- 4) 二叉树的测试数据用文本文件方式给出，例如测试数据名为 `bt151.btr` 的二叉树，测试数据名为 `tree10.tre` 的树或森林，可参考发来的二叉树与树（森林）形状和参考存储文件；
- 5) 二叉树创建方法可自行选择；
- 6) 程序运行、测试正确；
- 7) 实验程序有较好可读性，各运算和变量的命名直观易懂，符合软件工程要求；
- 8) 程序有适当的注释，程序的书写要采用缩进格式。

3 实验任务

编写算法求解下列问题：

- 1) 将二叉链表存储的二叉树转换为顺序存储形式（提示：数组中要扩展为完全二叉树）。
实验测试数据基本要求：
第一组数据： `bt8.btr`
第二组数据： `bt14.btr`
- 2) 键盘输入一个元素 `x`，求其父结点、兄弟结点、子结点的值，不存在时给出相应提示信息。对兄弟结点和孩子结点，存在时要明确指出是左兄弟、左孩子、右兄弟或右孩子。实验测试数据基本要求：
第一组数据： `bt8.btr`
第二组数据： `bt21.btr`
- 3) 对二叉链表表示的二叉树，求 2 个结点最近共同祖先。实验测试数据基本要求：
第一组数据： `bt261.btr`
第二组数据： `bt21.btr`
- 4) 输出二叉树从每个叶子结点到根结点的路径（经历的结点）。实验测试数据基本要求：
第一组数据： `bt261.btr`
第二组数据： `bt21.btr`
- 5) 树（森林）以孩子兄弟链表存储，求树（森林）的度。实验测试数据基本要求：
第一组数据： `tree11.tre`
第二组数据： `f20.tre`

6) 树（森林）以孩子兄弟链表存储，输出广义表表示的树（森林）。

例对图 1 所示森林，输出为：A(B(E(K),F,G),C(H,I),D(J)), L(M,N), O(P)

实验测试数据基本要求：

第一组数据： tree11.tre

第二组数据： f20.tre

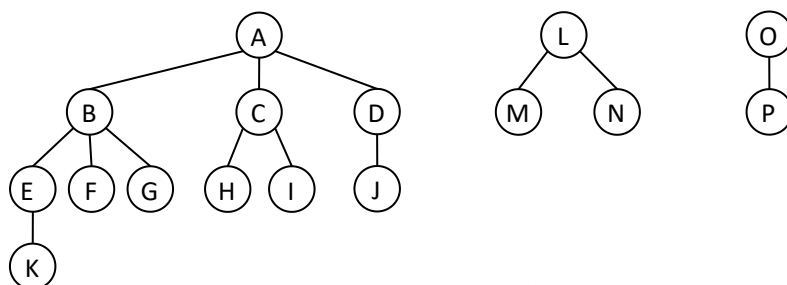


图 1 一个森林示意图

4 算法设计与实现描述

在正式实现算法之前，程序首先通过独立的头文件封装了二叉树的二叉链表存储及树（森林）的双亲存储与孩子兄弟链表存储结构。其中 CreateBiTree.h 提供了二叉树的结点定义、队列辅助结构、层次遍历、键盘交互创建、完全二叉树创建以及从数据文件创建二叉树的完整功能。createTree.h 则提供了树（森林）的双亲表示法定义、孩子兄弟表示法定义、从文件创建双亲表示树、以及从双亲表示转换为孩子兄弟链表的功能。这些底层基础设施为后续六道实验题的求解提供了健壮的数据结构支撑。

【头文件 1: CreateBiTree.h —— 二叉树二叉链表存储及创建算法】

```
#ifndef CREATE_TREE_H
#define CREATE_TREE_H

#include <iostream>

#include <cstring> // 字符串处理函数 strlen, strtok, strstr 等函数所在头文件
#include <cstdio> // 补充: 用于 FILE, fopen, fgets, printf 等文件 I/O 函数

//树（森林）的双亲表示定义和算法
#define MAXLEN 100

typedef char elementType;

//树的结点结构
typedef struct pNode
{
    elementType data; //结点数据域
    int parent; //父节点索引
}PTNode;

//树的双亲表示存储结构
typedef struct pTree
```

```
{
    PTNode node[MAXLEN]; //树的结点数组
    int n;                //树中结点数
}pTree;
//初始化树
void initialTree(pTree &T)
{
    T.n=0; //初始化树中结点数为0
}
//求祖先
bool getAncestor(pTree &T, elementType x)
{
    int w=0;
    elementType y;
    y=x;

    for(w=0;w<T.n;w++)
    {
        if(T.node[w].data==y)
        {
            w=T.node[w].parent; //找到x的双亲节点
            y=T.node[w].data;
            std::cout<<y<<"\t";
            break;
        }
    }
    if(w>=T.n) //x不在树中, 返回false
        return false;

    //继续求x的双亲节点的双亲节点
    while(w!=-1)
    {
        if(T.node[w].data==y)
        {
            w=T.node[w].parent; //找到w的双亲节点
            y=T.node[w].data;
            std::cout<<y<<"\t";
        }
        else
            w=(w+1)%T.n;
    }
    return true;
}
//求孩子
```

```

void getChildren(pTree &T, elementType x)
{
    int i,w;

    for(w=0;w<T.n;w++)    //找到 x 在结点数组中的下标
    {
        if(T.node[w].data==x)
            break;
    }
    if(w>=T.n)    //x 不在树中
        return;
    for(i=0;i<T.n;i++)
    {
        if(T.node[i].parent==w)    //输出所有孩子结点
            std::cout<<T.node[i].data<<"\t";
    }
    std::cout<<std::endl;
}

//先根遍历
int firstChild(pTree &T,int v)    //求下标为 v 的结点的第一个孩子的下标
{
    int w;
    if(v==-1)
        return -1;
    for(w=0;w<T.n;w++)
    {
        if(T.node[w].parent==v)
            return w;
    }
    return -1;
}

int nextSibling(pTree &T,int v,int w)    //求结点 v 的下标为 w 的孩子结点的下一个兄弟的下标
{
    int i;
    for(i=w+1;i<T.n;i++)
        if(T.node[i].parent==v)
            return i;
    return -1;
}

void preOrder(pTree &T,int v)//先根遍历以节点 v 为根的子树
{
    int w;
    std::cout<<T.node[v].data<<"\t";

```

```

w=firstChild(T,v);

```

```

        while(w!=-1)
        {
            preOrder(T,w);

            w=nextSibling(T,v,w);
        }
    }
}

void preTraverse(pTree &T)//先根遍历整个树（森林）T
{
    int i;

    int visited[MAXLEN];

    for(i=0;i<T.n;i++)
    {
        visited[i]=0;
    }

    //遍历每个根节点（双亲为-1的结点）
    for(i=0;i<T.n;i++)
    {
        if(T.node[i].parent==-1)
            preOrder(T,i);
    }
}

void postOrder(pTree &T,int v)//后根遍历以节点 v 为根的子树
{
    int w;

    w=firstChild(T,v);

    while(w!=-1)
    {
        postOrder(T,w);

        w=nextSibling(T,v,w);
    }

    std::cout<<T.node[v].data<<"\t";    //访问根节点
}

void postTraverse(pTree &T)//后根遍历整个树（森林）T
{
    int i;

    int visited[MAXLEN];

    for(i=0;i<T.n;i++)
    {
        visited[i]=0;
    }

    //遍历每个根节点（双亲为-1的结点）
    for(i=0;i<T.n;i++)
    {
        if(T.node[i].parent==-1)

```

```

        postOrder(T,i);
    }
}
//打印树
void printTree(pTree &T)
{
    int i;

    std::cout<<"下标\t 数据\t 双亲"<<std::endl;

    for(i=0;i<T.n;i++)

        std::cout<<i<<"\t"<<T.node[i].data<<"\t"<<T.node[i].parent<<std::endl;
}

//孩子兄弟表示法的定义-----
//从文件创建树（双亲表示法）的函数-----
//孩子兄弟表示法的存储结构
typedef char elementType;
typedef struct csNode
{
    elementType data;

    struct csNode *firstChild, *nextSibling;
}csNode,*csTree;
//去掉字符串左边的空格
void strLTrim(char* str)
{
    int i,j;

    int n=0;

    n=strlen(str)+1;

    for(i=0;i<n;i++)

    {

        if(str[i]!=' ') //找到第一个非空格字符的位置

            break;

    }

    //把字符串左移，去掉左边空格

    for(j=0;j<n;j++)

    {

        str[j]=str[i];

        i++;

    }

}

//***** 从文件创建树（双亲表示法）*****//
/* 函数功能：从文本文件创建树 */
/* 参数： char fileName[]: 文件名 */
/* 返回值： pTree &T: 生成的树 */
/* 返回值： bool 类型， true 表示创建成功， false 表示创建失败 */
/* 函数名称： CreateTreeFromFile(const char fileName[], pTree &T) */

```

```
/** 主要步骤：从文件中读取树结构数据并建立双亲表示法的树 */
//*****//

int CreateTreeFromFile(const char fileName[], pTree &T)//从文件创建树（双亲表示法）
{
    FILE* pFile;    //文件指针变量

    char str[1000]; //用于存放文件读取的一行字符串
    char strTemp[10]; //临时字符串
    int i=0,j=0;

    pFile=fopen(fileName,"r");
    if(!pFile)
    {
        printf("打开文件%s 失败! \n",fileName);
        return false;
    }

    while(fgets(str,1000,pFile)!=NULL) //跳过前面的注释行
    {
        //去掉字符串左边的空格
        strLTrim(str);

        if (str[0]=='\n') //空行，继续读取下一行
            continue;

        strncpy(strTemp,str,2);

        if(strstr(strTemp,"//")!=NULL) //注释行
            continue;

        else //不是空行或注释行，结束循环
            break;
    }

    //检查字符串 str 中是否包含"Tree or Forest"
    if(strstr(str,"Tree or Forest")==NULL)
    {
        printf("文件格式不正确! \n");
        fclose(pFile); //关闭文件
        return false;
    }

    //读取下一行数据字符串 str
    while(fgets(str,1000,pFile)!=NULL)
    {
        //去掉字符串左边的空格
        strLTrim(str);

        if (str[0]=='\n') //空行，继续读取下一行
            continue;

        strncpy(strTemp,str,2);

        if(strstr(strTemp,"//")!=NULL) //如果是注释行，则继续读取下一行
            continue;
    }
}
```

```
        else //如果是数据行，则跳出循环
            break;
    }

    //处理结点数据行
    char* token=strtok(str, " ");
    int nNum=0;
    while(token!=NULL)
    {
        T.node[nNum].data=*token;

        T.node[nNum].parent=-1; //暂时将双亲节点设为-1

        token = strtok( NULL, " ");

        nNum++;
    }

    //处理边的数据行
    int nP; //父节点下标
    int nC; //子节点下标
    elementType Nf,Ns; //父节点和子节点数据
    while(fgets(str,1000,pFile)!=NULL)
    {
        //去掉字符串左边的空格
        strLTrim(str);

        if (str[0]=='\n') //空行，继续读取下一行
            continue;

        strncpy(strTemp,str,2);
        if(strstr(strTemp,"//")!=NULL) //如果是注释行，则继续读取下一行
            continue;

        char* token=strtok(str, " "); //将 str 按空格分割，依次取出父节点和子节点

        if(token==NULL) //如果没有数据，则退出
        {
            printf("文件数据格式错误! \n");
            fclose(pFile); //关闭文件
            return false;
        }
        Nf=*token; //父节点数据

        token = strtok( NULL, " "); //取出第二个字符串（子节点）
        if(token==NULL) //如果没有数据，则退出
        {
            printf("文件数据格式错误! \n");
```



```

        fclose(pFile); //关闭文件
        return false;
    }

    Ns=*token; //得到子节点数据

    //查找父节点下标
    for(nP=0;nP<nNum;nP++)
    {
        if(T.node[nP].data==Nf) //在节点列表中查找父节点
            break;
    }

    //查找子节点下标
    for(nC=0;nC<nNum;nC++)
    {
        if(T.node[nC].data==Ns) //在节点列表中查找子节点
            break;
    }

    T.node[nC].parent=nP; //nC 的双亲节点为 nP
}

T.n=nNum; //设置树的结点数
fclose(pFile); //关闭文件
return true;
}

//返回结点下标为 w 的结点的下一个兄弟结点的下标
int next(pTree T,int w)
{
    int i;
    for(i=w+1;i<T.n;i++)
    {
        if(T.node[w].parent==T.node[i].parent)
            return i;
    }

    return -1;
}

//根据双亲表示法创建孩子兄弟表示法
void create(csNode *&T,pTree &T1,int v)
{
    int w;

    T=new csNode;
    T->data=T1.node[v].data;
    T->firstChild=NULL;
    T->nextSibling=NULL;

    w=firstChild(T1,v); //找到 v 的第一个孩子
    if(w!=-1)
    {

```

```

        create(T->firstChild,T1,w);
    }

    w=next(T1,v);        //找到 v 的下一个兄弟
    if(w!=-1)
    {
        create(T->nextSibling,T1,w);
    }
}

//根据双亲表示法创建孩子兄弟链表
void createCsTree(csNode *&T,pTree T1)
{
    int i;
    //找到 T1 的根节点
    for(i=0;i<T1.n;i++)
    {
        if(T1.node[i].parent==-1)    //双亲为-1 的为根结点
            break;
    }

    if(i<T1.n)
        create(T,T1,i);
}

//孩子兄弟表示法相关定义和算法-----
#endif // CREATE_TREE_H

```

【头文件 2: createTree.h —— 树（森林）双亲表示与孩子兄弟链表存储及创建算法】

```

#ifndef CREATEBITREE_H
#define CREATEBITREE_H

#define NODENUM 100    //定义最大结点数

#include <iostream>
#include <cstring>
#include <cstdio>
using namespace std;

// 二叉树节点定义
typedef char elementType;
typedef struct btNode {
    elementType data;
    struct btNode *lChild, *rChild;
} btNode;

```

```

typedef char eleType;

```

```

// 队列定义（简单版本）

```

```
typedef struct {  
    btNode* data[100];  
    int front, rear;  
} seqQueue;
```

```
// 队列操作函数
```

```
void initialQueue(seqQueue *Q) { Q->front = Q->rear = 0; }  
void enqueue(seqQueue *Q, btNode *x) { Q->data[Q->rear++] = x; }  
int queueEmpty(seqQueue Q) { return Q.front == Q.rear; }  
void getFront(seqQueue Q, btNode* &x) { x = Q.data[Q.front]; }  
void outQueue(seqQueue *Q, btNode* &x) { x = Q->data[Q->front++]; }  
void strLTrim(char* str); //申明删除字符串左边空格
```

```
//先序层次遍历，BFS--因为结构定义及运算实现的次序关系，临时放在此处
```

```
void hieTraverse(btNode *T)  
{  
    btNode *p;  
    seqQueue Q;  
    initialQueue(&Q);  
    enqueue(&Q,T);  
    while(!queueEmpty(Q))  
    {  
        getFront(Q,p);//获取队头元素  
        cout<<p->data<<" ";          //访问根结点。打印当前结点元素值，替代visit(T)函数  
        if(p->lChild)  
            enqueue(&Q,p->lChild);  
        if(p->rChild)  
            enqueue(&Q,p->rChild);  
        outQueue(&Q,p); //出队  
    }  
}
```

```
//键盘交互创建二叉树开始-----
```

```
//键盘交互递归创建二叉树子树函数
```

```
void createSubTree(btNode *&p, int k)  
{  
    //k=1--左子树；k=2--右子树  
    btNode *s;  
    elementType x;  
    cout<<"请输入结点元素数值， '/'表示无子树，x=";  
    cin>>x;  
    if(x!='/')  
    {  
        s=new btNode;
```

```

        s->data=x;

        s->lChild=NULL;

        s->rChild=NULL;

        if(k==1)

            p->lChild=s;    //s 接到 p 的左孩子

        if(k==2)

            p->rChild=s;    //s 接为 p 的右孩子

        createSubTree(s,1); //递归创建 s 的左子树

        createSubTree(s,2); //递归创建 s 的右子树

    }

}

```

```

//键盘交互创建二叉树主控函数
void createBTConsole(btNode *&T)
{
    btNode *p;
    elementType x;

    cout<<"请按先序序列输入二叉树结点，（'/'表示无子树）:"<<endl;

    cout<<"请输入结点元素数值， '/'表示无子树，x=";

    cin>>x;

    if(x=='/')

    {

        T = NULL; // 增加容错：如果是空树也要赋 NULL

        return;    //空树，退出

    }

    T=new btNode;

    T->data=x;

    T->lChild=NULL;

    T->rChild=NULL;

```

```

        createSubTree(T,1);    //创建根结点的左子树

        createSubTree(T,2);    //创建根结点的右子树

    }

```

//键盘交互创建二叉树开始-----

```

//键盘交互完全二叉树方式创建二叉树开始-----
void getCompleteArr(elementType *arr, int &num)
{
    //完全二叉树顺序存储方式创建二叉链表结构二叉树

    //缺少的结点数值用'/'代替

    //用'#'结束结点输入

    //arr 为结点数组

    //num 返回实际结点数

    elementType x;

```

```

int i=1;    //arr[0]单元不用

num=0;

cout<<"请按完全二叉树方式输入二叉树结点， '/'表示缺少结点， '#'结束输入。"<<endl;

cout<<"请输入结点元素数值， '/'表示缺少结点， '#'结束输入， x=";

cin>>x;

while(x!='#')
{
    *(arr+i)=x;

    i++;

    num++;

```

```

    cin>>x;

}

}

```

//从完全二叉树顺序存储方式的数组创建二叉树

```

void createBtArr(btNode* &T, elementType* InArr, int i, int n)

```

```

{
    //T--为根结点指针

    //InArr--为按完全二叉树存储的树的结点数组

    //i--当前结点序号

    //n--二叉树结点总数

    // 必须明确给 T 赋空值，否则会产生野指针导致遍历崩溃
    if(i > n || InArr[i] == '/')
    {
        T = NULL;

        return;

    }

```

// 有效数据创建结点

```

T=new btNode;        //创建根结点

T->data=InArr[i];    //结点赋值

T->lChild=NULL;

T->rChild=NULL;

```

```

createBtArr(T->lChild, InArr, 2*i, n);    //递归创建 T 的左子树

```

```

createBtArr(T->rChild, InArr, 2*i+1, n); //递归创建 T 的右子树

```

```

}

```

//键盘交互完全二叉树方式创建二叉树结束-----

//数据文件创建二叉树开始-----

//从文本文件数据读入到数组中，同时返回实际结点数量

```

bool ReadFileToArray(const char fileName[], char strLine[NODENUM][3], int & nArrLen)

```

```

{

```

```
FILE* pFile;      //定义二叉树的文件指针

char str[1000];    //存放读出一行文本的字符串

char strTemp[10]; //判断是否注释行
```

```
pFile=fopen(fileName,"r");
if(!pFile)
{
    cout<<"错误: 文件"<<fileName<<"打开失败。"<<endl;
    return false;
}
```

```
while(fgets(str,1000,pFile)!=NULL) //跳过空行和注释行
{
    //删除字符串左边空格

    strlTrim(str);

    if (str[0]=='\n') //空行, 继续读取下一行
        continue;
```

```
    strncpy(strTemp,str,2);

    strTemp[2] = '\0'; // 增加安全截断

    if(strstr(strTemp,"//")!=NULL) //跳过注释行
        continue;

    else //非注释行、非空行, 跳出循环
        break;
}

//循环结束, str 中应该已经是二叉树数据标识"BinaryTree", 判断文件格式
if(strstr(str,"BinaryTree")==NULL)
{
    printf("错误: 打开的文件格式错误! \n");
    fclose(pFile); //关闭文件
    return false;
}
```

```
nArrLen=0; //数组行号初始化为 0

while(fgets(str,1000,pFile)!=NULL)
{
    //删除字符串左边空格

    strlTrim(str);

    if (str[0]=='\n') //空行, 继续读取下一行
        continue;

    strncpy(strTemp,str,2);

    strTemp[2] = '\0';

    if(strstr(strTemp,"//")!=NULL) //注释行, 跳过, 继续读取下一行
```

```
continue;
```

```
char* token=strtok(str," \r\n"); // 以空格和换行符为分隔符，更加安全

if(token==NULL) //分割为空串，失败退出
{
    printf("错误：读取二叉树结点数据失败！\n");
    fclose(pFile); //关闭文件
    return false;
}

strLine[nArrLen][0]=*token; //获取结点数据

token = strtok( NULL, " \r\n"); //读取下一个子串，即此结点的左子树信息
if(token==NULL)
{
    printf("错误：读取二叉树结点数据失败！\n");
    fclose(pFile);
    return false;
}

strLine[nArrLen][1]=*token; //获取此结点的左子树信息数据

token = strtok( NULL, " \r\n"); //读取下一个子串，即此结点的右子树信息
if(token==NULL)
{
    printf("错误：读取二叉树结点数据失败！\n");
    fclose(pFile);
    return false;
}

strLine[nArrLen][2]=*token; //获取此结点的右子树信息数据

nArrLen++; //数组行号加1
}

fclose(pFile); //关闭文件

return true;
}

//从数组创建二叉树--数组中保存的是二叉树的先序序列，及每个结点的子树信息
bool CreateBiTreeFromFile(btNode* & pBT, char strLine[NODENUM][3],int nLen, int & nRow)
{
    if((nRow>=nLen) || (nLen==0)) {
        pBT = NULL; // 同样增加容错赋空
        return false;
    }

    //根据数组数据递归创建子树

    pBT=new btNode;
    pBT->data=strLine[nRow][0];
    pBT->lChild=NULL;
```

```

pBT->rChild=NULL;

int nRowNext=nRow; //保留本次递归的行号
if(strLine[nRowNext][1]=='1') //当前结点有左子树，递归创建左子树，读下一行数据
{
    nRow++;
    CreateBiTreeFromFile(pBT->lChild, strLine,nLen,nRow);
}
if(strLine[nRowNext][2]=='1') //当前结点有右子树，递归创建右子树，读下一行数组
{
    nRow++;
    CreateBiTreeFromFile(pBT->rChild, strLine,nLen,nRow);
}
return true;
}
//数据文件创建二叉树结束-----
//删除字符串、字符数组左边空格
void strLTrim(char* str)
{
    int i,j;
    int n=0;
    n=strlen(str)+1;
    for(i=0;i<n;i++)
    {
        if(str[i]!=' ') //找到左起第一个非空格位置
            break;
    }
    //以第一个非空格字符为首字符移动字符串
    if (i > 0) {
        for(j = 0; i < n; j++, i++)
        {
            str[j]=str[i];
        }
    }
}
#endif

```

（1）将二叉链表存储的二叉树转换为顺序存储形式

【算法思想】

借助层次遍历（BFS）逐层访问二叉树结点，利用完全二叉树的下标规律（根结点下标为 1，左孩子下标为 $2i$ ，右孩子下标为 $2i+1$ ）将结点映射到顺序存储数组中。使用队列同时存储结点指针及其对应下标，出队时将结点数据写入数组对应位置，并将其左右孩子（若存在）连同计算出的下标入队。遍历结束后，数组中未填充的位置表示完全二叉树中的空缺结点，以特殊符号‘/’标识。

【算法步骤】

①初始化：创建存储下标映射的队列，将根结点与下标 1 入队，初始化最大下标 `maxIdx` 为 1。

②层次遍历：当队列非空时，取出队头元素，将结点数据写入 `arr[cur.idx]`。

③左孩子处理：若左孩子存在，计算其下标 `leftIdx = cur.idx × 2`，将（左孩子，`leftIdx`）入队，更新 `maxIdx`。

④右孩子处理：若右孩子存在，计算其下标 `rightIdx = cur.idx × 2 + 1`，将（右孩子，`rightIdx`）入队，更新 `maxIdx`。

⑤输出结果：遍历 `arr[maxIdx]`，输出下标与对应的结点值。

【算法描述和实现】

```
// 将二叉链表存储的二叉树转换为完全二叉树顺序存储数组
// T: 二叉树根结点指针
// arr: 输出数组（下标从 1 开始，arr[0]弃用）
// maxIdx: 返回数组中最大下标
void BiTreeToArray(btNode* T, elementType* arr, int& maxIdx)
{
    if (T == NULL) // 空树，返回
    {
        maxIdx = 0;
        return;
    }

    // 使用队列进行层次遍历，队列元素存储（结点指针，完全二叉树下标）
    struct QNode
    {
        btNode* node; // 结点指针
        int idx; // 完全二叉树下标
    };
};
```

```
const int MAXQ = 1000;
QNode q[MAXQ];
int front = 0, rear = 0;
```

```
// 根结点入队，下标为 1
q[rear].node = T;
q[rear].idx = 1;
rear++;
maxIdx = 1; // 初始化最大下标为 1
```

```
while (front < rear)
{
    QNode cur = q[front]; // 获取队头元素
    front++;
```

```
// 将结点数据写入数组对应下标  
arr[cur.idx] = cur.node->data;
```

```
// 左孩子入队  
if (cur.node->lChild != NULL)  
{  
    int leftIdx = cur.idx * 2; // 左孩子下标为父结点下标的 2 倍  
    q[rear].node = cur.node->lChild;  
    q[rear].idx = leftIdx;  
    rear++;  
    if (leftIdx > maxIdx)  
        maxIdx = leftIdx;  
}
```

```
// 右孩子入队  
if (cur.node->rChild != NULL)  
{  
    int rightIdx = cur.idx * 2 + 1; // 右孩子下标为父结点下标的 2 倍 + 1  
    q[rear].node = cur.node->rChild;  
    q[rear].idx = rightIdx;  
    rear++;  
    if (rightIdx > maxIdx)  
        maxIdx = rightIdx;  
}  
}
```

```
// 打印顺序存储数组  
void printArray(elementType* arr, int maxIdx)  
{  
    cout << "完全二叉树顺序存储数组（下标从 1 开始）: " << endl;  
    for (int i = 1; i <= maxIdx; i++)  
    {  
        cout << i << ": " << arr[i];  
        if (i < maxIdx)  
            cout << " ";  
        if (i % 10 == 0) // 每 10 个换行  
            cout << endl;  
    }  
    cout << endl;  
}
```

(2) 查询结点 x 的父结点、兄弟结点、子结点

【算法思想】

采用“先定位目标，再反向推导关系”的两阶段策略。第一阶段通过递归遍历（getNode 函数）在二叉树中查找值为 x 的结点，若不存在则直接提示并返回。第二阶段通过递归查找父结点（getParent 函数）：从根开始，若当前结点的左孩子或右孩子为目标结点 x，则当前结点即为父结点。有了父结点后，兄弟关系和子结点关系可直接通过对比指针推导得出。

【算法步骤】

①查找目标：调用 getNode(T, x) 递归查找值为 x 的结点。若返回 NULL，提示结点不存在并结束。

②根结点判断：若 x 等于根结点数据，提示其为根结点，无父结点和兄弟结点。

③查找父结点：调用 getParent(T, x) 从根开始递归查找父结点，输出父结点值。

④推导兄弟：若父结点的左孩子为 x 且右孩子存在，则右孩子为右兄弟；若父结点的右孩子为 x 且左孩子存在，则左孩子为左兄弟；否则无兄弟。

⑤输出子结点：直接检查目标结点的 lChild 和 rChild 指针，输出存在的孩子结点。

【算法描述和实现】

```
// 辅助函数 1：在二叉树中递归查找值为 x 的结点本身
btNode* getNode(btNode* T, elementType x) {
    if (T == NULL) return NULL;
    if (T->data == x) return T; // 找到了

    btNode* p = getNode(T->lChild, x); // 去左子树找
    if (p != NULL) return p;
    return getNode(T->rChild, x); // 去右子树找
}

btNode* getParent(btNode* T, elementType x) {
    if (T == NULL) return NULL;

    // 如果当前结点的左孩子或右孩子是 x，那么当前结点就是 x 的父结点！
    if ((T->lChild != NULL && T->lChild->data == x) ||
        (T->rChild != NULL && T->rChild->data == x)) {
        return T;
    }

    // 否则继续往左子树深处找
    btNode* p = getParent(T->lChild, x);
    if (p != NULL) return p;

    // 左边没找到，再往右子树深处找
    return getParent(T->rChild, x);
}

void queryNodeInfo(btNode* T, elementType x) {
    if (T == NULL) {
        cout << "这是一棵空树！" << endl;
        return;
    }

    // 1. 先验证这个结点到底存不存在
```

```

btNode* target = getNode(T, x); // 查找值为 x 的结点

if (target == NULL) {
    cout << " 提示: 结点 '" << x << "' 不在这棵二叉树中" << endl;
    return;
}

// 2. 找父结点和兄弟结点
if (T->data == x) {
    cout << "结点 '" << x << "' 是根结点, 没有父结点和兄弟结点" << endl;
}
else {
    btNode* parent = getParent(T, x);
    cout << "结点 '" << x << "' 的父结点是: '" << parent->data << "'" << endl;
    // 兄弟结点: 如果父结点的左孩子是 x, 那么兄弟就是父结点的右孩子; 反之亦然
    if (parent->lChild != NULL && parent->lChild->data == x && parent->rChild != NULL) {
        cout << "结点 '" << x << "' 的右兄弟结点是: '" << parent->rChild->data << "'" << endl;
    }
    else if (parent->rChild != NULL && parent->rChild->data == x && parent->lChild != NULL) {
        cout << "结点 '" << x << "' 的左兄弟结点是: '" << parent->lChild->data << "'" << endl;
    }
    else {
        cout << "没有兄弟结点。" << endl;
    }
}

// 3. 找孩子结点
if (target->lChild != NULL) {
    cout << "结点 '" << x << "' 的左孩子是: '" << target->lChild->data << "'" << endl;
}
if (target->rChild != NULL) {
    cout << "结点 '" << x << "' 的右孩子是: '" << target->rChild->data << "'" << endl;
}
if (target->lChild == NULL && target->rChild == NULL){
    cout << "结点 '" << x << "' 没有孩子结点。" << endl;
}
}

```

(3) 求两个结点的最近公共祖先 (LCA)

【算法思想】

采用经典的后序遍历递归策略求解 LCA。从根结点出发, 若当前结点为 NULL 则返回 NULL; 若当前结点等于 x 或 y 则直接返回当前结点。否则分别在左子树和右子树中递归查找。根据左右子树返回结果的组合判断: 若左右均非空, 说明 x 和 y 分别位于当前结点的左右子树中, 当前结点即为 LCA; 若仅一侧非空, 说明两个结点都在该侧子树中, 向上传递该侧结果; 若两侧均为空, 返回 NULL。算法的时间复杂度为 $O(n)$, 空间复杂度 $O(h)$ (h 为树高)。

【算法步骤】

①递归出口：若 T 为 NULL，返回 NULL；若 T->data 等于 x 或 y，返回 T。
②递归左子树：leftResult = getancestor(T->lChild, x, y)。
③递归右子树：rightResult = getancestor(T->rChild, x, y)。
④结果判定：若 leftResult 和 rightResult 均非空，返回 T（当前结点为 LCA）；若仅 leftResult 非空则返回 leftResult；若仅 rightResult 非空则返回 rightResult；否则返回 NULL。

⑤主调验证：调用 getNode 验证 x 和 y 是否均在树中，验证通过后调用 getancestor 输出结果。

【算法描述和实现】

```
// 辅助函数在二叉树中递归查找值为 x 的结点本身
btNode* getNode(btNode* T, elementType x) {
    if (T == NULL) return NULL;
    if (T->data == x) return T; // 找到了

    btNode* p = getNode(T->lChild, x); // 去左子树找
    if (p != NULL) return p;
    return getNode(T->rChild, x); // 去右子树找
}

btNode* getancestor(btNode* T, elementType x, elementType y) {
    if (T == NULL) return NULL;
    if (T->data == x || T->data == y) return T; // 找到了

    btNode* lefttree = getancestor(T->lChild, x, y); // 去左子树找
    btNode* righttree = getancestor(T->rChild, x, y); // 去右子树找
    if (lefttree != NULL && righttree != NULL)
        return T;
    if (lefttree != NULL) return lefttree;
    if (righttree != NULL) return righttree;
    return NULL;
}

void queryNodeInfo(btNode* T, elementType x, elementType y) {
    if (T == NULL) {
        cout << "这是一棵空树!" << endl;
        return;
    }
    // 1. 先验证这个结点到底存不存在
    btNode* target = getNode(T, x); // 查找值为 x 的结点
    if (target == NULL) {
        cout << "提示: 结点 '" << x << "' 不在这棵二叉树中" << endl;
        return;
    }
    target = getNode(T, y); // 查找值为 y 的结点
    if (target == NULL) {
        cout << "提示: 结点 '" << y << "' 不在这棵二叉树中" << endl;
    }
}
```

```

        return;
    }

    if (getancestor(T, x,y) != NULL)// 如果最近公共祖先存在
    {
        cout << "结点 '" << x << "' 和结点 '" << y << "' 的最近公共祖先是: '" << getancestor(T, x,y)->data << "' "
        << endl;
    }
}

```

(4) 输出每个叶子结点到根结点的路径

【算法思想】

使用 DFS 深度优先遍历二叉树，在递归过程中维护一个 path 数组记录从根到当前结点的路径。关键设计在于 pathLen 参数采用值传递（非引用传递），这使得递归返回时 pathLen 自动恢复，天然实现回溯效果，无需手动撤销操作。当遍历到叶子结点（左右孩子均为 NULL）时，逆序输出 path 数组即可得到从叶子到根的路径。

【算法步骤】

- ①递归出口：若 T 为 NULL，直接返回。
- ②记录当前结点：path[pathLen] = T->data, pathLen++。
- ③叶子判定：若 T->lChild 和 T->rChild 均为 NULL，逆序输出 path[0..pathLen-1] 作为从叶子到根的路径。
- ④递归子树：若非叶子，分别递归遍历左子树和右子树（pathLen 按值传递自动回溯）。

【算法描述和实现】

```

// 核心算法：DFS 查找并打印所有叶子结点到根结点的路径
// 这里的 pathLen 利用值传递实现天然的路径回溯
void printLeafToRootPaths(btNode* T, elementType path[], int pathLen) {
    if (T == NULL) return; // 空结点直接返回

    // 1. 将当前结点存入路径数组，并将路径长度 + 1
    path[pathLen] = T->data;
    pathLen++;
}

```

```

// 2. 判断当前结点是不是“叶子结点”（左右孩子都为空）
if (T->lChild == NULL && T->rChild == NULL) {
    cout << "叶子结点 '" << T->data << "' 到根的路径: ";

    // 3. 题目要求“从叶子到根”，所以我们把 path 数组
    for (int i = pathLen - 1; i >= 0; i--) {
        cout << path[i];
        if (i > 0) cout << " -> ";
    }
    cout << endl;
}
else {
    printLeafToRootPaths(T->lChild, path, pathLen);
}

```

```

        printLeafToRootPaths(T->rChild, path, pathLen);
    }
}

```

(5) 求孩子兄弟链表存储的树（森林）的度

【算法思想】

树的度定义为树中所有结点的最大孩子数。在孩子兄弟表示法中，一个结点的所有孩子通过 firstChild 指针指向第一个孩子，然后通过 nextSibling 指针串联。因此统计一个结点的孩子数等价于遍历其 firstChild 出发的 nextSibling 链。算法采用递归策略：对每个结点，统计孩子个数 childCount，同时递归求各子树的最大度 subDegree，最终返回 childCount 与 subDegree 中的最大值。对于森林，遍历每棵树的根（通过 nextSibling 连接），取各树度的最大值。

【算法步骤】

- ①getDegree 函数：若 T 为 NULL 返回 0；初始化 maxDegree=0，childCount=0。
- ②遍历孩子链：从 child = T->firstChild 开始，对每个 child 执行 childCount++，递归调用 subDegree = getDegree(child)更新 maxDegree，child 移向下一个兄弟。
- ③取最大值：若 childCount > maxDegree 则更新 maxDegree，返回 maxDegree。
- ④getForestDegree 函数：遍历森林中各树的根（p 从 forest 开始沿 nextSibling 移动），对每棵树调用 getDegree(p)并记录全局最大值。

【算法描述和实现】

```

// 求孩子兄弟链表表示的树（森林）的度
// 对当前结点 T，统计其孩子个数，并递归求各子树的度，返回最大值
int getDegree(csNode* T)
{
    if (T == NULL)
        return 0;

    int maxDegree = 0;    // 记录当前树/森林的最大度
    int childCount = 0;   // 当前结点的孩子个数

    csNode* child = T->firstChild;
    while (child != NULL)
    {
        childCount++;      // 每遇到一个孩子，计数+1
        // 递归求以该孩子为根的子树中的最大度
        int subDegree = getDegree(child); // 递归求子树的度
        if (subDegree > maxDegree) maxDegree = subDegree;

        child = child->nextSibling;    // 移向下一个兄弟（即下一个孩子）
    }

    // 当前结点的孩子数与已求出的子树最大度比较
    if (childCount > maxDegree)
        maxDegree = childCount;

    return maxDegree;
}

// 求森林（多棵树）的度：遍历森林中每棵树，取最大度
int getForestDegree(csNode* forest)

```

```

{
    if (forest == NULL)
        return 0;

    int maxDegree = 0;

    // 森林由多棵树通过 nextSibling 连接
    // 遍历森林中的每棵树的根
    csNode* p = forest;
    while (p != NULL)
    {
        int degree = getDegree(p); // 求以 p 为根的树的度
        if (degree > maxDegree)
            maxDegree = degree;

        p = p->nextSibling; // 下一棵树
    }

    return maxDegree;
}

```

(6) 输出孩子兄弟链表存储的树（森林）的广义表形式

【算法思想】

广义表表示法将树的结构用括号和逗号进行嵌套描述，格式为“根（子树 1，子树 2，...）”。在孩子兄弟表示法中，firstChild 指向第一个孩子（对应广义表中的括号内部），nextSibling 指向下一个兄弟（对应广义表中的逗号分隔）。算法递归输出：先输出当前结点数据；若存在 firstChild，则输出“（”，递归输出孩子子树，再输出“）”；若存在 nextSibling，则输出“，”，递归输出兄弟子树。

【算法步骤】

①递归出口：若 T 为 NULL 直接返回。

②输出结点：cout << T->data。

③ 处理孩子：若 T->firstChild 存在，输出“（”，递归调用 printgeneraliedlist(T->firstChild)，输出“）”。

④ 处理兄弟：若 T->nextSibling 存在，输出“，”，递归调用 printgeneraliedlist(T->nextSibling)。

⑤主调：分别读入 tree11.tre 和 f20.tre，构建孩子兄弟链表后调用递归输出。

【算法描述和实现】

```

void printgeneraliedlist(csNode* T)
{
    if (T == NULL)
        return;

    // 1. 打印当前结点
    cout << T->data;

    // 2. 如果有孩子，说明有子树，用一对括号把子树包起来
    if (T->firstChild != NULL) {
        cout << "(";
        printgeneraliedlist(T->firstChild); // 递归打印孩子
        cout << ")";
    }

    if (T->nextSibling != NULL) {
        cout << ",";
        printgeneraliedlist(T->nextSibling); // 递归打印兄弟
    }
}

```



```

    }

    // 3. 如果有亲兄弟，说明是同级，用逗号隔开
    if (T->nextSibling != NULL) {
        cout << ",";

        printgeneraliedlist(T->nextSibling); // 递归打印兄弟
    }
}
}

```

5 运行结果截图及说明

(1) 二叉链表转换为完全二叉树顺序存储

程序分别读入 bt8.btr 和 bt14.btr 两个测试数据文件，先输出二叉树的层次遍历序列以验证建树正确性，再调用 BiTreeToArray 函数完成转换并打印顺序存储数组。转换后的数组中，空缺位置以 '/' 填充，下标严格遵循完全二叉树的编号规则（根为 1，左孩子为 $2i$ ，右孩子为 $2i+1$ ）。

```

=====
测试第一组数据: bt8.btr
=====
二叉树的层次遍历结果: 1 2 8 3 6 4 5 7
转换为顺序存储形式 (扩展为完全二叉树):
完全二叉树顺序存储数组 (下标从1开始):
1:1  2:2  3:8  4:3  5:6  6:/  7:/  8:4  9:5  10:/
11:7

=====
测试第二组数据: bt14.btr
=====
二叉树的层次遍历结果: A B C I D F J E G K L H M N
转换为顺序存储形式 (扩展为完全二叉树):
完全二叉树顺序存储数组 (下标从1开始):
1:A  2:B  3:/  4:C  5:I  6:/  7:/  8:D  9:F  10:J
11:/  12:/  13:/  14:/  15:/  16:/  17:E  18:G  19:/  20:K
21:L  22:/  23:/  24:/  25:/  26:/  27:/  28:/  29:/  30:/
31:/  32:/  33:/  34:/  35:/  36:/  37:H  38:/  39:/  40:/
41:/  42:M  43:/  44:/  45:/  46:/  47:/  48:/  49:/  50:/
51:/  52:/  53:/  54:/  55:/  56:/  57:/  58:/  59:/  60:/
61:/  62:/  63:/  64:/  65:/  66:/  67:/  68:/  69:/  70:/
71:/  72:/  73:/  74:/  75:/  76:/  77:/  78:/  79:/  80:/
81:/  82:/  83:/  84:N

```

(2) 查询结点的父结点、兄弟结点、子结点

程序分别读入 bt8.btr 和 bt21.btr，提示用户键盘输入待查询的结点值 x。对于存在的结点，程序输出其父结点、兄弟结点（明确左/右）以及孩子结点（明确左/右）；对于不存在的结点，给出相应提示；对于根结点，说明其无父结点和兄弟结点。

```

=====
测试第一组数据: bt8.btr
=====
请输入要查询的结点值: x=3
结点 '3' 的父结点是: '2'
结点 '3' 的右兄弟结点是: '6'
结点 '3' 的左孩子是: '4'
结点 '3' 的右孩子是: '5'

=====
测试第二组数据: bt21.btr
=====
请输入要查询的结点值: x=o
结点 'o' 的父结点是: 'k'
结点 'o' 的左兄弟结点是: 'l'
结点 'o' 的左孩子是: 'p'
结点 'o' 的右孩子是: 'r'

```

(3) 求两个结点的最近公共祖先

程序分别读入 bt261.btr 和 bt21.btr，针对指定的结点对（第一组 'g' 和 'u'，第二

组'e'和'h')调用 getAncestor 函数递归查找 LCA。算法从根结点开始,利用后序遍历自底向上汇总信息,当左右子树各找到一个目标结点时,当前结点即为最近公共祖先。

```
=====
测试第一组数据: bt261.btr
=====
结点 'g' 和结点 'u' 的最近公共祖先是: 'a'

=====
测试第二组数据: bt21.btr
=====
结点 'e' 和结点 'h' 的最近公共祖先是: 'b'
```

(4) 输出每个叶子结点到根结点的路径

程序分别读入 bt261.btr 和 bt21.btr,通过 DFS 递归遍历二叉树,在递归过程中将途经结点存入 path 数组。当遇到叶子结点(左右孩子均为空)时,逆序输出 path 数组中的结点序列,形成从叶子到根的路径。

```
=====
测试第一组数据: bt261.btr
=====
叶子结点 'e' 到根的路径: e -> d -> c -> b -> a
叶子结点 'f' 到根的路径: f -> d -> c -> b -> a
叶子结点 'h' 到根的路径: h -> g -> c -> b -> a
叶子结点 'k' 到根的路径: k -> j -> i -> b -> a
叶子结点 'm' 到根的路径: m -> l -> i -> b -> a
叶子结点 'q' 到根的路径: q -> p -> o -> n -> a
叶子结点 'r' 到根的路径: r -> p -> o -> n -> a
叶子结点 's' 到根的路径: s -> o -> n -> a
叶子结点 'v' 到根的路径: v -> u -> t -> n -> a
叶子结点 'w' 到根的路径: w -> u -> t -> n -> a
叶子结点 'y' 到根的路径: y -> x -> t -> n -> a
叶子结点 'z' 到根的路径: z -> x -> t -> n -> a

=====
测试第二组数据: bt21.btr
=====
叶子结点 'e' 到根的路径: e -> d -> c -> b -> a
叶子结点 'f' 到根的路径: f -> d -> c -> b -> a
叶子结点 'g' 到根的路径: g -> c -> b -> a
叶子结点 'i' 到根的路径: i -> h -> b -> a
叶子结点 'j' 到根的路径: j -> h -> b -> a
叶子结点 'm' 到根的路径: m -> l -> k -> a
叶子结点 'n' 到根的路径: n -> l -> k -> a
叶子结点 'q' 到根的路径: q -> p -> o -> k -> a
叶子结点 's' 到根的路径: s -> r -> o -> k -> a
叶子结点 'u' 到根的路径: u -> t -> r -> o -> k -> a
```

(5) 求树(森林)的度

程序分别读入 tree11.tre 和 f20.tre,先从文件构建双亲表示法的树,再转换为孩子兄弟链表表示。getDegree 函数递归遍历每个结点,统计其孩子个数,并与各子树的度取最大值。getForestDegree 函数遍历森林中每棵树,返回全局最大度。

```
=====
测试第一组数据: tree11.tre
=====
树(森林)的度为: 3
=====
测试第二组数据: f20.tre
=====
树(森林)的度为: 3
```

(6) 输出树(森林)的广义表形式

程序分别读入 tree11.tre 和 f20.tre,构建孩子兄弟链表后调用 printgeneraliedlist 函数递归输出广义表。输出格式遵循:结点数据→若有孩子则加括号递归输出→若有兄弟则加逗号继续输出。多棵树的森林之间以逗号分隔。

```
=====
测试第一组数据: tree11.tre
=====
```

```
A(B(E(K),F),C(G),D(H,I,J))
=====
```

```
测试第二组数据: f20.tre
=====
```

```
A(B(E,F),C(G),D(H,I,J)),K(L,M(N)),O(P(Q(S(T))),R)
=====
```

6 总结、心得和建议

本次实验围绕二叉树与树（森林）的存储表示、遍历算法及典型应用展开，涵盖了二叉链表到顺序存储的转换、结点关系查询、最近公共祖先查找、叶子结点到根路径输出、树（森林）的度计算以及广义表输出等六个综合性任务。通过这些实践，对二叉树和树的递归结构有了更深刻的理解。

【心得】

（1）层次遍历在完全二叉树映射中的妙用：第 1 题要求将二叉链表转换为完全二叉树的顺序存储形式。核心难点在于如何将任意形态的二叉树映射到完全二叉树的下标体系。通过层次遍历配合队列结构，利用完全二叉树父结点下标为 i 时左孩子为 $2i$ 、右孩子为 $2i+1$ 的性质，可以优雅地完成映射。缺少结点的位置用特殊符号 '/' 填充，保证了数组索引与完全二叉树逻辑位置的一一对应。这个过程让我深刻体会到：完全二叉树的顺序存储之所以高效，正是因为其下标规律消除了指针开销，而队列恰好能按层序逐层推进，两者结合相得益彰。

（2）递归查找与关系推理的工程实践：第 2 题要求查询某结点的父结点、兄弟结点和子结点。程序先通过递归遍历定位目标结点，再通过判断当前结点的左右孩子是否为目标值的方式逆向查找父结点，进而推导出兄弟关系。这种先定位目标，再反向推导关系的两阶段设计，将复杂的关系查询拆解为独立的查找与判定步骤，体现了分而治之的工程思维。值得注意的边界情况包括：根结点无父结点和兄弟结点、目标结点不存在时的提示反馈等，这些边界处理是程序健壮性的关键。

（3）最近公共祖先算法的递归精粹：第 3 题求两结点的最近公共祖先，采用了经典的后序遍历递归策略：若当前结点等于 x 或 y 则返回当前结点；否则分别在左右子树中递归查找；若左右子树均返回非空，说明当前结点即为 LCA；若仅一侧非空则向上传递该侧结果。这个算法的时间复杂度为 $O(n)$ ，空间复杂度取决于递归深度。它完美展示了递归自底向上汇总信息的能力，每个子树向父结点报告「我这里面有没有 x 或 y 」，父结点汇总左右子树信息后做出决策。这种思想在图论和竞赛编程中极为常见，是数据结构课程的核心收获之一。

（4）路径回溯与值传递的巧妙运用：第 4 题输出叶子到根的路径，采用 DFS 深度优先遍历，利用数组 `path` 记录当前路径上的结点序列。关键技巧在于 `pathLen` 参数采用值传递（而非引用传递），这实现了天然的回溯——从递归返回时，`pathLen` 自动恢复到进入前的值，无需手动撤销操作。这一细节设计使得代码简洁且不易出错，充分体现了对 C++ 参数传递机制的深刻理解。逆序输出 `path` 数组即可得到「从叶子到根」的路径，符合题目要求。

（5）孩子兄弟链表的多重遍历融合：第 5 题和第 6 题使用孩子兄弟表示法处理树和森林。求树的度时，需要同时做两件事：统计当前结点的孩子个数，并递归求各子树的最大度，最终取两者的最大值。这种边遍历边统计的模式，正是在递归过程中携带额外状态信息的典型应用。输出广义表时，通过先输出结点数据、若有孩子则加括号递归输出、若有兄弟则加逗号继续输出的三段式逻辑，完美模拟了广义表的嵌套结构。这两

道题让我深入理解了左孩子右兄弟表示法如何将任意树转化为二叉树，以及在这种结构上如何设计遍历算法。

【建议】

（1）引入非递归遍历的对比实践：本次实验的算法大多基于递归实现，递归虽然简洁优雅，但在极端深度下存在栈溢出风险。建议在后续实验中增设非递归（迭代）遍历的专项训练，引导学生使用显式栈模拟递归过程，加深对系统调用栈和手动栈管理的理解。

（2）增加算法复杂度分析环节：目前实验报告侧重于功能实现，建议要求学生每个算法明确给出时间复杂度和空间复杂度分析。例如 LCA 算法虽然代码简洁，但其在多次查询场景下效率不如 Tarjan 离线算法或倍增法。引入复杂度分析有助于培养算法选型的工程意识。

(附录, 提交报告时去掉这部分内容)

一、二叉树二叉链表存储创建和销毁参考

1. 数据文件输入创建二叉树

推荐一种基于文本文件的二叉树创建方法, 即将二叉树的结点信息保存在一个文本文件中, 然后用程序自动读入来创建二叉树。我们先来定义文本文件的格式。

标识行: **BinaryTree**—标识这是一个二叉树的数据文件。

结点行: 每个结点一行, 结点严格按照先序遍历次序排列。每行 3 列。第 1 列为结点数据; 第 2 列标识有无左子树, 1—有左子树, 0—无左子树; 第 3 列标识有无右子树, 1—有右子树, 0—无右子树。

对如图所示二叉树, 完整数据文件如下:

BinaryTree

a 1 1

b 1 1

d 0 0

e 1 1

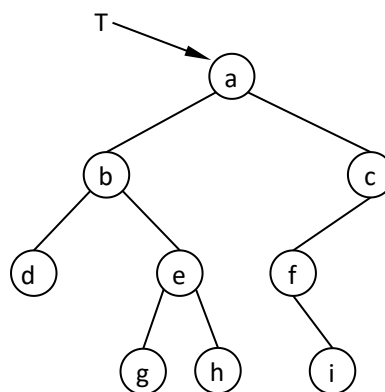
g 0 0

h 0 0

c 1 0

f 0 1

i 0 0



图一棵二叉树

数据文件的扩展名随意, 只要按文本文件读写即可, 例如上述文件不妨命名为 **BiTree9.btr**。

接下来介绍基于上述数据文件创建二叉树的函数。

【结点数据读入数组函数】

因为是按先序次序递归创建二叉树, 在创建过程中又要记录读取数据的行数, 直接从文件中读取数据创建时处理不方便, 所以我们先把文件中的数据读取到一个二维数组中, 再从数组中读取数据来创建二叉树。从文件读取数据到数组的函数代码如下:

```
bool ReadFileToArray(char fileName[], char strLine[100][3], int & nArrLen)
{
    // fileName[]存放文件名
    // strLine[][3]存放结点的二维数组, 数组的 3 列对应数据文件的 3 列
    // nArrLen 返回二叉树结点的个数
    FILE* pFile; //定义二叉树的文件指针
    char str[1000]; //存放读出一行文本的字符串
    pFile=fopen(fileName,"r");
    if(!pFile)
    {
        printf("二叉树数据文件打开失败! \n");
        return false;
    }
    //读取文件第 1 行, 判断二叉树标识 BinaryTree 是否正确
    if(fgets(str,1000,pFile)!=NULL)
    {
        if(strcmp(str,"BinaryTree\n")!=0)
```

```

        {
            printf("打开的文件格式错误! \n");
            fclose(pFile);    //关闭文件
            return false;
        }
    }
    nArrLen=0;
    while(nArrLen<100&&fscanf(pFile,"%c %c %c\n",
        &strLine[nArrLen][0],&strLine[nArrLen][1],&strLine[nArrLen][2])!=EOF)
    {
        //循环读取结点行数据，存入数组，结点总数加 1
        nArrLen++;
    }
    fclose(pFile); //关闭文件
    return true;
}
【从数组数据按先序次序创建二叉树】
bool CreateBiTreeFromFile(BiNode* & pBT, char strLine[100][3],int nLen, int & nRow)
{
    //strLine[100][3]--保存结点数据的二维数组
    //nLen--结点个数
    //nRow--数组当前行号
    if((nRow>=nLen) || (nLen==0))
        return false;    //数据已经处理完毕，或者没有数据，退出
    //根据数组数据递归创建二叉树
    pBT=new BiNode;    //建立根结点
    pBT->data=strLine[nRow][0];
    pBT->lChild=NULL;
    pBT->rChild=NULL;
    int nRowNext=nRow;    //保留本次递归的行号
    if(strLine[nRowNext][1]=='1')
    {
        //当前结点有左子树，读下一行数据，递归调用创建左子树。
        nRow++;    //行号加 1
        CreateBiTreeFromFile(pBT->lChild, strLine,nLen,nRow);
    }
    if(strLine[nRowNext][2]=='1')
    {
        //当前结点有右子树，读下一行数据，递归调用创建右子树。
        nRow++;    //行号加 1
        CreateBiTreeFromFile(pBT->rChild, strLine,nLen,nRow);
    }
    return true;
}

```

2. 二叉树的销毁

创建了二叉链表存储结构的二叉树，使用完毕后应当释放此二叉树占用的内存，因为在创建二叉树时使用 malloc()函数或 new 操作符动态申请了内存，当这个二叉树不再需要时，

必须手工释放动态申请的内存，否则造成内存泄漏。下面给出销毁二叉链表存储结构二叉树的程序。

```
void DestroyBiTree(BiNode* pBT)
{
    if(pBT)
    {
        DestroyBiTree(pBT->lChild);    //递归销毁左子树
        DestroyBiTree(pBT->rChild);    //递归销毁右子树
        delete pBT;                    //释放当前根结点
        pBT=NULL;                      //置空指针
    }
}
```

二、树（森林）创建和销毁参考

(1)树（森林）的创建

本实验提供的创建代码，创建二叉链表表示的树（森林）分为 2 个步骤，第一步：读取文本文件，创建双亲表示的树（森林）；第二步：从双亲表示转换为二叉链表表示的树（森林）。

(2)树（森林）数据文件格式说明

数据文件主要包含三个部分：树（森林）标识；结点列表；父子结点对（边）。

①标识行

Tree or Forest，以区别其它数据文件，这一行是非必须的。

②结点列表

给出树（森林）中的所有结点，结点次序无关，只要列出所有结点即可。如图 7-1 所示的森林，结点列表可为：

//下面为树（森林）的结点列表

A B C D E F G H I J K L M N O P

③父子结点对（边）信息

父子对信息严格按照父结点、子结点表示一对父子结点，父子对也次序无关，只要列出森林中所有父子对即可，例图 7-1 所示森林，所有父子对为：

//以下为父子结点对（边）信息

A B

A C

A D

B E

B F

B G

C H

C I

D J

E K

L M

LN
OP

(3)创建树（森林）包含文件说明

`createTree.h`，包括树（森林）的双亲存储、二叉链表存储的定义；从文件创建双亲表示的树（森林）；从双亲表示的森林创建二叉链表表示的森林；其它辅助算法。